Algoritmi di crittografia: Domande orale

Due on Thursday, January 09, 2024

Last update: 17:42pm

Bruno Pesole

Question 1

Diffie-Hellman?

Le applicazioni moderne hanno il problema della gestione delle chiavi e dunque Diffie e Hellman hanno studiato un algoritmo che possa permettere di usare chiavi differenti: quella di cifratura pubblica e quella di decifrazione privata. Questo algoritmo funziona così:

1. A e B si accordano sul $\mod p$ (numero primo) e un generatore g in Z_p^*. Il tutto pubblicamente
2. A sceglie a casa un $a \in Z_p^*$ privata e calcola $x_a = g^a \mod p$ e lo invia a B
3. B fa la stessa cosa scegliendo un $b \in Z_p^*$ e calcola $x_b = g^b \mod p$ e lo invia ad A
4. B calcola $z_b = x_a^b \mod p$
5. A calcola $z_a = x_b^a \mod p$

Il segreto condiviso sarà $z = z_a = z_b$. Questa uguaglianza si dimostra semplicemente

$$
\begin{aligned}
z_a &= x_b^a \mod p && (1)\\
&= (g^b \mod p)^a \mod p && (2)\\
&= (g^a)^b \mod p && (3)\\
&= (g^a \mod p)^b \mod p && (4)\\
&= x_a^b \mod p = z_b && (5)
\end{aligned}
$$

Per quanto riguarda l'**efficienza** sappiamo che tutte le quantità necessarie ad A per completare il protocollo sono calcolabili efficientemente; mentre le computazioni più onerose sono i due esponenziali modulari (hanno complessità polinomiale nella lunghezza dei numeri). Mentre l'operazione di scelta di p (con relativa scelta di g) viene fatta adoperando i safe-prime, ovvero quei primi che si possono esprimere come $p = 2q + 1$ dove q è un numero primo
La **sicurezza** invece riguarda la difficoltà di calcolare z conoscendo solo p, g, x_a e x_b perché l'unico modo per risalire a z è il calcolo del logaritmo discreto (poiché
$x_a = g^a \mod p \Rightarrow a = \log_g x_a \mod p$)

Question 2 continued on next page...

Question 2

Attacchi al protocollo

Un attacco vincente per questo protocollo è **Authenticated DH** (man-in-the-middle) in cui il malintenzionato può fingersi A o B per partecipare allo scambio di chiavi (una volta con A e l'altra con B) per poi decifrare i messaggi prima di reinviarli. A questo punto il malintenzionato può decidere se portare attacchi a lungo o breve termine. Per risolvere tale vulnerabilità si può utilizzare un'autenticazione da parte di entrambi: quindi una delle due deve possedere una chiave pubblica che viene usata dall'altra parte per verificarne la firma. Per fare ciò serve un long-term secret. E questo protocollo fornisce forward-secrecy, per cui nel caso si decifrasse un messaggio non si avrebbe modo di decifrare anche i precedenti

Question 3

Firma digitale?

La firma digitale è il metodo di autenticazione basato su crittografia a chiave pubblica, cioè una cifratura effettuata con una chiave privata, per cui l'autenticità del mittente deriva dal fatto che il messaggio può essere decifrato solo con la corrispondente chiave pubblica. Si possono utilizzare diversi algoritmi di cifratura per la firma, come ad esempio **RSA** grazie al ruolo simmetrico di e e d, e il messaggio viene poi cifrato con la chiave segreta, La sicurezza sta nel fatto che è computazionalmente intrattabile il problema di creare $(\bar{M}, \bar{F})$ dove $\bar{F} = \bar{M}^d \mod n$ senza conoscere d, In concreto si utilizzano delle funzioni hash per ottenere dei fingerprint e ridurre la dimensione dei numeri alla quale viene applicato il calcolo

1. A calcola $H = hash(M)$
2. Calcola $F = H^d \mod n$
3. Invia la coppia (M, F) a B
4. B si procura (e, n) da A
5. Calcola $H' = F^e \mod n$
6. Calcola $H = hash(M)$
7. Accetta solo se $H' = H$

Dato che la sequenza di bit è molto inferiore, lo spazio rimanente viene riempito con bit casuali (padding) e quindi il calcolo di RSA viene applicato a $M+$padding
Un altro algoritmo utilizzato è **ElGamal** che da sicurezza grazie alla presumibile intrattabilità del problema del calcolo del logaritmo discreto

Question 3 continued on next page...

1. A sceglie k t.c. $MCD(k, p-1) = 1$
2. Calcola $r = g^k \mod p$ e $s = k^{-1}(M - ar) \mod (p-1)$
3. Invia la coppia $(M, (r, s))$ a B
4. B si procura (p, g, A) da A
5. Calcola $x_1 = A^r \cdot r^s \mod p$ e $x_2 = g^M \mod p$
6. Accetta solo se $x_1 = x_2$

Per dimostrare la correttezza si verifica a cosa corrisponde $r^s \mod p$

$$
\begin{aligned}
r^s \mod p &= (g^k \mod p)^s \mod p && (6)\\
&= (g^k)^{k^{-1}(M-ar) \mod (p-1)} \mod p \Rightarrow (g^k)^{k^{-1}(M-ar)-(p-1)h} \mod p && (7)\\
&= g^{M-ar}((g^{-kh})^{p-1} \mod p) \mod p \Rightarrow g^M g^{-ar} \mod p && (8)\\
&= g^M (g^{-a} \mod p)^r \mod p && (9)\\
&= g^M A^{-r} \mod p && (10)
\end{aligned}
$$

Per cui è evidente che

$$
\begin{aligned}
x_1 &= A^r \cdot r^s \mod p && (11)\\
&= A^r \cdot (r^s \mod p) \mod p && (12)\\
&= \cancel{A^r} \cdot (g^M \cancel{A^{-r}} \mod p) \mod p = g^M \mod p && (13)\\
& && (14)
\end{aligned}
$$

Infine un altro algoritmo è **DSA**

1. A genera un primo q di 160 bit
2. Genera un primo p di 1024 bit t.c. q sia un divisore primo di $p - 1$
3. Determina $g \in Z_p^-$ di ordine q
4. Sceglie a caso un $a \in Z_q^*$ tenendolo privato
5. Calcola $A = G^a \mod p$
6. Pubblica (p, q, g, A)
7. Dato M, calcola $m =$SHA-1(M) di 160 bit
8. Sceglie k a caso in Z_q^*
9. Calcola $r = (g^k \mod p) \mod q$ e $s = k^{-1}(m + ar) \mod q$
10. Se r o s sono uguali a 0, allora si sceglie un k diverso, altrimenti si invia $(m, (r, s))$
11. B si procura la chiave pubblica (p, q, g, A)
12. Controlla che $r > 0$ e $s < q$
13. Calcola $x =$SHA-1$(M) \cdot (s^{-1} \mod q)$

Question 3 continued on next page...

14. Calcola $y = r \cdot (s^{-1} \mod q)$
15. Controlla se $(g^x A^y \mod p) \mod q = r$, se è corretto accetta la firma come autentica

La sicurezza sta proprio nel fatto che il calcolo del logaritmo discreto su un sottogruppo di q elementi sia molto difficile poiché il protocollo originale fornisce $\log(\sqrt{2^{160}}) = 80$ bit di sicurezza

Question 4

Come facciamo ad essere sicuri che la chiave appartenga effettivamente al mittente?

Quando B recupera la chiave pubblica di A deve essere sicuro che sia effettivamente la sua e per fare ciò si utilizzano protocolli come TLS/openSSL e GNUPG/openPGP. Ad esempio **TLS** viene utilizzato anche per le vendite online. Il server deve rispondere con un certificato di autenticità (una chiave pubblica del server firmata da un'autoriytà riconosciuta che funge da garante). A questo punto se il client possiede questo certificato, si procede con il protocollo concordato; se invece il client non ha il certificato ma possiede la chiave pubblica verificata dall'autorità che gestisce il server, il client acquisisce il certificato e lo salva per utilizzi futuri; Se invece nella peggiore delle ipotesi il client non possieda né il certificato, né la chiave pubblica verificata, allora dovrà autenticare quest'ultima mediante un'autorità a livello maggiore. Viene da se, che ogni cliente deve avere già delle chiavi pubbliche delle autorità di massimo livello.
Invece l'approccio **GNUPG** è di tipo P2P quindi il tutto si basa sulla fiducia

1. Un mittente di mail firma i messaggi per garantire la provenienza delle mail
2. Un destinatario consulta un archivio delle chiavi PGP per recuperare la chiave pubblica in questione
3. La chiave ha anche un fingerprint comunicato preventivamente in altri modi
4. A questo punto si può importare questa chiave nel proprio keyring attribuendo un livello di confidenza elevato
5. Se un destinatario non ha ricevuto il fingerprint, nota che altre persone hanno attribuito un'alta confidenza e questo è sufficiente per credere che quella chiave appartenga a quel mittente
6. A questio punto anche lui la importa nel proprio keyring attribuendo un livello di confidenza un po' meno elevato

Question 5

Accorgimenti firma digitale?

Per la sicurezza della firma digitale bisogna avere un paio di accorgimenti

1. **Il valore di k deve essere usato una sola volta** poiché, dato che vale $x_1 = x_2$ allora vale anche $\log_g x_1 \mod p = \log_g x_2 \mod p = M \mod p$ e si dimostra facilmente come

$$\begin{aligned} \log_g x_1 \mod p &= \log_g A^r \cdot r^s \mod p && (15)\\ &= (r \log_g A + s \log_g r) \mod p && (16)\\ &= (r \cdot a + s \cdot k) \mod p = M \mod p && (17) \end{aligned}$$

   In questo modo l'attaccante utilizza M_1 e M_2 in un sistema per ricavare k e a
   $\begin{cases} ar_1 + ks_1 = M_1(\mod p) \\ ar_2 + ks_2 = M_2(\mod p) \end{cases}$
   Quindi se k è diverso, il numero di equazioni è di uno in più e quindi il sistema diventa indeterminato

2. **Il protocollo deve essere utilizzato con l'hash del messaggio**, altrimenti si può apportare facilmente un message forgery attack, ovvero con lo scopo di produrre un messaggio correttamente firmato senza conoscere la chiave privata. Questo attacco utilizza una preventiva determinazione dei valori r e s. L'attacco si esegue scegliendo x, y t.c. $MCD(y.p-1) = 1$, poi si pone

$$r = g^x \cdot A^y \mod p = g^x \cdot (g^a \mod p)^y \mod p \Rightarrow g^{x+ay} \mod p$$

$$s = -r \cdot y^{-1} \mod (p-1)$$

   Da qui poi si può derivare M di cui (r, s) ne è la firma.
   Questo si dimostra in questo modo

$$\begin{aligned} A^r r^s \mod p &= (g^a \mod p)^r (g^{x+ay} \mod p)^s \mod p && (18)\\ &= g^{ar+xs+ays^{(*)}} \mod p \Rightarrow g^{xs} \mod p && (19)\\ &= g^{M+h(p-1)} \mod p = g^M \mod p && (20) \end{aligned}$$

   (*): sappiamo che
   $y \cdot s = -r + h'(p-1)$, dunque $g^{ar+ays} \mod p = g^{ar-ar+ah'(p-1)} \mod p = 1$
   Quindi per evitare questo attacco, si firma l'hash del messaggio anziché il messaggio stesso $H(M||r)$, in questo modo l'attaccante non può porre
   $M = x \cdot s \mod (p-1)$ poiché B non verifica questo e dunque l'attaccante dovrebbe trovare un M t.c. $H(M||r) = x \cdot s \mod (p-1)$ ma H non dovrebbe essere first-image resistant

Question 6

Vantaggi nell'usare curve ellittiche?

Le curve ellittiche offrono diversi vantaggi in crittografia rispetto ad altri metodi. Alcuni di questi vantaggi includono:

1. **Livello di sicurezza elevato:** Le curve ellittiche possono fornire un elevato livello di sicurezza con chiavi più corte rispetto ad altri algoritmi crittografici, come RSA. Ciò è particolarmente importante nelle applicazioni in cui le risorse sono limitate, come dispositivi mobili.
2. **Efficienza computazionale:** Le operazioni aritmetiche su curve ellittiche sono computazionalmente più efficienti rispetto a molte altre tecniche crittografiche, come la fattorizzazione di numeri in RSA. Di conseguenza, le curve ellittiche richiedono meno risorse computazionali, rendendole adatte per dispositivi con limitazioni di potenza di calcolo.
3. **Dimensioni delle chiavi più piccole:** Le chiavi utilizzate nelle curve ellittiche sono generalmente più corte rispetto a chiavi con livelli di sicurezza equivalenti in altri sistemi crittografici. Ciò si traduce in una maggiore efficienza nelle operazioni crittografiche e nella riduzione dello spazio di archiviazione richiesto.
4. **Resistenza agli attacchi quantistici:** Le curve ellittiche sembrano essere meno vulnerabili agli attacchi basati su computer quantistici rispetto ad alcuni altri algoritmi crittografici tradizionali. Ciò è importante considerando lo sviluppo di computer quantistici che potrebbero minare la sicurezza di algoritmi basati sulla fattorizzazione o sulla logaritmo discreto.
5. **Flessibilità nelle dimensioni della chiave:** Le curve ellittiche offrono una maggiore flessibilità nelle dimensioni della chiave, consentendo la scelta di curve con parametri appropriati per soddisfare requisiti specifici di sicurezza e prestazioni.
6. **Algoritmi di firma digitali efficienti:** Le firme digitali basate su curve ellittiche sono più efficienti rispetto a quelle basate su altri algoritmi, come le firme digitali basate su RSA.

Ci sono anche degli svantaggi però

1. **Complessità di implementazione:** L'implementazione corretta delle curve ellittiche richiede una comprensione approfondita della teoria matematica alla base di queste curve.
2. **Standardizzazione e interoperabilità:** Diversi standard per le curve ellittiche possono causare problemi di interoperabilità tra sistemi diversi.
3. **Mancanza di trasparenza:** Alcune implementazioni di curve ellittiche potrebbero mancare di trasparenza, compromettendo la sicurezza del sistema.
4. **Dimensioni della chiave e parametri critici:** La sicurezza delle curve ellittiche dipende dalla corretta scelta dei parametri critici.
5. **Attacchi laterali e implementazioni non sicure:** Le implementazioni reali delle curve ellittiche possono essere vulnerabili ad attacchi laterali o implementazioni non sicure.

Question 7

RSA?

RSA basa la sua sicurezza sulla difficoltà della fattorizzazione dei numeri interi. Questo protocollo ha un solo parametro in input che è la lunghezza in bit dell'intero su cui si effettuano le riduzioni modulari

1. A genera due primi p e q a caso di lunghezza $\frac{N}{2}$ bit
2. Calcola $n = pq$
3. Calcola $\phi(n) = (p-1)(q-1)$
4. Ricava e t.c. $MCD(e, \phi(n)) = 1$
5. Calcola d inverso moltiplicativo di $e \mod \phi(n)$
6. Diffonde (e, n) come chiave pubblica e mantiene d come chiave segreta
7. B si procura (e, n)
8. Calcola $C = M^e \mod n$
9. Invia C ad A come messaggio cifrato
10. Per decifrare il messaggio, A calcola $M = C^d \mod n$

Per dimostrare la correttezza di M

$$C^d \mod n = (M^e \mod n)^d \mod n \Rightarrow (M^{ed}) \mod n \tag{21}$$
$$= M^{k\phi(n)+1} \mod n \Rightarrow M^{k\phi(n)}M \mod n \tag{22}$$
$$= M \cdot M^{k(p-1)(q-1)} \mod n \Rightarrow_{TCR} M \cdot M^{k(p-1)(q-1)} \mod p \tag{23}$$
$$= M(M^{p-1})^{k(q-1)} \mod p \Rightarrow M(M^{p-1} \mod p)^{k(q-1)} \mod p \tag{24}$$
$$= M \cdot 1^{k(q-1)} \mod p = M \mod p \tag{25}$$

Stessa cosa per $M^{k(p-1)(q-1)} \mod q = M \mod q$, quindi dato che $n = pq$, per il TCR è dimostrato
Questo è un protocollo oneroto in termini di efficienza a causa della scelta di e, nonostante p e q possano essere scelti molto velocemente data l'abbondanza dei numeri primi
A livello di sicurezza, la difficoltà risiede appunto nella difficoltà a fattorizzare numeri di grandi dimensioni

Question 9 continued on next page...

Question 8

Che problemi ha

I problemi che potrebbe avere sono

1. **Malleabilità**: ovvero che da un testo cifrato $C = E(M_1)$ si possa creare un C_2 t.c. $C" = E(M_2)$ con M_2 strettamente in relazione con M_1
2. e **troppo piccolo**: Per questo esiste il FIPS che prevede un $e > 2^{16} + 1$
3. **Possibilità di fattorizzare** n: se p e q sono troppo vicini, si può rendere molto più semplice la fattorizzazione di n
4. **Vulnerabilità dipendenti da generatori casuali**: Da uno studio è scaturito che su circa 5mln di RSA-1024 bit i duplicati erano frequenti poiché avevano un fattore primo in comune

Question 9 continued on next page...

Question 9

Attacchi al protocollo

Di seguito alcuni attacchi al protocollo

1. Sia $C_1 = M_1^e \mod n$ un testo cifrato, si crea un testo cifrato intermedio come $C_I = 2^e \mod n$. A questo punto si può creare $C_2 = C_1 \cdot C_I$, dunque se in qualche modo si riesce a far decifrare C_2 si può facilmente ottenere M_1 perché

$$C_2^d \mod n = (C_1 C_I)^d \mod n \tag{26}$$
$$= ((M_1^e \mod n)(2^e \mod n))^d \mod \tag{27}$$
$$= ((M_1^e)^d (2^e)^d) \mod n = (M_1 \cdot 2) \mod n \tag{28}$$

2. Se e è troppo piccolo, la riduzione $\mod n$ è inutile e quindi $C = M^e$, per cui il malintenzionato può semplicemente calcolare la radice e-esima per trovare M

3. Se p e q sono valori troppo vicini si può riuscire a ricavarli in quanto il prodotto di due numeri primi si può esprimere come differenza di quadrati: $p = x + y$ e $q = x - y$ e quindi $n = x^2 - y^2 = (x + y)(x - y)$
   Per effettuare un attacco RSA tramite fattorizzazione si vogliono trovare le quantità $\frac{p+q}{2}$ e $\frac{p-q}{2}$ senza conoscere p e q tramite brute force. Per fare ciò si utilizza la riscrittura $n = x^2 - y^2 \Rightarrow y^2 = x^2 - n$, e dato che $p \neq q$ e $n = pq$ allora $\frac{p+q}{2} > \sqrt{n}$ e pertanto il punto di partenza potrebbe essere $\lceil \sqrt{n} \rceil$

   (a) Dato n, poniamo $x = \lceil \sqrt{n} \rceil$
   (b) Calcoliamo $z = x^2 - n$
   (c) Se x non è un quadrato perfetto, si incrementa di 1 e torna al passo (b), altrimenti va avanti
   (d) Calcola e restituisci $x + \sqrt{z}$ e $x - \sqrt{z}$

   Per capire quanto devono essere vicini questi valori per apportare l'attacco, supponiamo che valga $\frac{p-q}{2} < c \cdot \sqrt{2q}$ allora dall'uguaglianza $(\frac{p+q}{2})^2 - n = (\frac{p-q}{2})^2$ possiamo ricavare $(\frac{p+q}{2} + \sqrt{n})(\frac{p+q}{2} - \sqrt{n}) = (\frac{p-q}{2})^2$. Dato che $q < \sqrt{n} < p$ possiamo minorare con $2q$ il primo fattore a sx sostituendo p e $\sqrt{n}$ con q, e possiamo maggiorare il membro di dx con $2qc^2$ ottenendo $\cancel{2}q(\frac{p+q}{2} - \sqrt{n}) < \cancel{2}qc^2$ e questo è il numero di iterazioni che l'algoritmo usa per arrivare alla soluzione. Dato che $\mathcal{O}(\sqrt{n})$ allora p e q non devono coincidere nella metà più significativa ($\mathcal{O}(\sqrt[4]{n})$)

Question 10

Rabin?

Rabin è molto simile a RSA ma con il limite per cui la decifrazione produce quattro possibili candidati e, se il testo originale è un numero oppure una sequenza di bit si incorre in problemi. L'algoritmo opera in questo modo

1. A genera due numeri primi a caso $p \equiv 3 \mod 4$ e $q \equiv 3 \mod 4$ di lunghezza $\frac{N}{2}$ bit
2. Calcola $n = pq$
3. Diffonde n come chiave pubblica e tiene p e q come segreti
4. B si procura n
5. Calcola $C = M^2 \mod n$
6. Invia C come messaggio cifrato

La decifrazione avviene in questo modo (ricordando che se p divide n, allora $(x \mod n) \mod p = x \mod p$)

1. Si calcolano $M_p = C^{\frac{p+1}{4}} \mod p$ e $M_q = C^{\frac{q+1}{4}} \mod q$ e si dimostra in questo modo

$$
\begin{aligned}
M_p^2 \mod p &= (C^{\frac{p+1}{4}})^2 \mod p \Rightarrow C^{\frac{p+1}{2}} \mod p && (29)\\
&= (C^{\frac{p-1}{2}} \cdot C) \mod p && (30)\\
&= ((M^2 \mod n)^{\frac{p-1}{2}} \cdot C) \mod p && (31)\\
&= (((M^2 \mod n) \mod p)^{\frac{p-1}{2}} \cdot C) \mod p && (32)\\
&= ((M^2 \mod p)^{\frac{p-1}{2}} \cdot C) \mod p && (33)\\
&= (M^{p-1} \cdot C) \mod p \Rightarrow ((M^{p-1} \mod p) \cdot C) \mod p && (34)\\
&= (1 \cdot C) \mod P = C \mod p && (35)
\end{aligned}
$$

   (Analogamente $M_q^2 \mod q = C \mod q$)

2. Calcola le seguenti 4 quantità (ciascuna una radce quadrata di $C \mod n$)

   (a) $M_1 = (q \cdot (q^{-1} \mod p))M_p + (p \cdot (p^{-1} \mod q))M_q \Rightarrow (C_pM_p + C_qM_q) \mod n$
   (b) $M_2 = n - M_1 \Rightarrow (-C_pM_p - C_qM_q) \mod n$
   (c) $M_3 = (q \cdot (q^{-1} \mod p))M_p - (p \cdot (p^{-1} \mod q))M_q \Rightarrow (C_pM_p - C_qM_q) \mod n$
   (d) $M_4 = n - M_3 \Rightarrow (-C_pM_p + C_qM_q) \mod n$

   Una di queste è quella originale
   La dimostrazione per M_1 (analoga M_3)

$$
\begin{aligned}
M_1^2 \mod p &= ((q \cdot (q^{-1} \mod p))M_p + ((p \cdot (p^{-1} \mod q))M_q) \mod n)^2 \mod p && (36)\\
&= (M_p \mod p)^2 \mod p && (37)\\
&= M_p^2 \mod p = C \mod p && (38)
\end{aligned}
$$

Question 10 continued on next page...

Dal TCR, dato che anche $M_1^2 \mod q = C \mod q$, allora $M_1^{"} \mod n = C \mod n$
Per dimostrare invece M_2 (analoga M_4)

$$M_2^2 \mod n = (n - M_1)^2 \mod n \tag{39}$$
$$= M_1^2 \mod n = C \mod n \tag{40}$$

Question 11

Perché serve l'associatività nei gruppi?

In crittografia, l'associatività è una proprietà importante che spesso è richiesta nelle operazioni di combinazione di elementi di un gruppo. Un gruppo in crittografia è un insieme di elementi dotato di un'operazione binaria che soddisfa quattro proprietà fondamentali: chiusura, associatività, esistenza di un elemento neutro e esistenza di elementi inversi.
L'associatività è la proprietà che afferma che l'ordine in cui si eseguono le operazioni binarie in un gruppo non influisce sul risultato finale. In altre parole, se hai tre elementi a,b e c appartenenti al gruppo, l'associatività afferma che $(a \cdot b) \cdot c)$ è uguale a $a \cdot (b \cdot c)$, indipendentemente da come vengono raggruppati gli elementi.
Questa proprietà è cruciale perché consente di definire operazioni di combinazione di elementi in modo coerente e prevedibile. Nei contesti crittografici, dove si manipolano dati sensibili e si effettuano operazioni matematiche su di essi, è importante garantire la coerenza e la prevedibilità delle operazioni. L'associatività contribuisce a garantire che le operazioni siano ben definite e che non dipendano dall'ordine delle operazioni.
Ad esempio, l'operazione di moltiplicazione in molti gruppi crittografici è associativa. Se si considera l'esponenziazione modulare in crittografia, l'associatività è una proprietà fondamentale, altrimenti non funzionerebbe né DH, né l'algoritmo veloce per l'esponenziale modulare

Question 12

Rho per logaritmo discreto?

Per calcolare il logaritmo discreto si utilizza un algoritmo di brute force chiamato "baby-steps giant-steps" che consente di stabilire se privilegiare lo spazio o il tempo. Basta conoscere p e il generatore per poi procedere calcolando due successioni di numeri: una con incrementi piccoli e l'altra con incrementi più grandi; a quel punti, il termine che appare in entrambe le successioni, consente di calcolare il logaritmo. Si scelgano dunque due valori r e s t.c. $r \cdot s \geq p$
NOTA: Sappiamo che un numero in Z_p può essere espresso come
$t = j + i \cdot r$ con $j = 0, 1, ..., r - 1$ e $i = 0, 1, ..., s - 1$ per cui i
baby steps saranno $1, g, g^2, ..., g^{r-1}$ e i giant steps $x, xg^{-r}, xg^{-2r}, ..., xg^{-(s-1)r}$. A questo punto, se per qualche valore di i, j risulta $g^i = xg^{-jr} \Rightarrow g^{i+jr} = x \Rightarrow i + jr = \log_g x$. Qui la correttezza è dimostrata dal fatto che ogni numero in Z_p può essere espresso come $i + jr$
L'algoritmo funziona memorizzando i termini baby-steps in una lookup table T, che poi viene confrontata con i termini della giant-steps cercando una collisione. Il tutto viene eseguito

Question 12 continued on next page...

in $\mathcal{O}(r+s)$, e dato che $r \cdot s \geq p$, il minimo si ottiene ponendo $r = s = \lceil \sqrt{p} \rceil$ (Quindi $\mathcal{O}(\sqrt{p})$). Lo spazio invece è $\mathcal{O}(r)$

Question 13

Rho di Pollard?

L'algoritmo di fattorizzazione ρ di Pollard, a partire da $x_1 \in Z_n$, produce una successione di valori $x_{j+1} = f(x_j) \mod n$ che dovrebbe avere una proprietà di sequenza casuale che ad un certo punto inizierà a ripetersi. Le iterate che vengono salvate sono quelle con indice $i = 2^t$ (quindi raddoppia ogni volta). Questo algoritmo non termina mai, oppure se termina restituisce un risultato non banale di n. Dato che $m = MCD(y - x_j, n)$ allora vuol dire che y e x_j sono congrui modulo m. In breve, lo scopo dell'algoritmo, è di trovare coppie di iterate diverse ma congrue modulo un divisore di n
Per ogni fattore q di n esiste una successione ombra $\{z_i\}_{i \geq 1}$, ottenuta riducendo ogni termine x modulo q. Si può dimostrare che la successione ombra ha la stessa struttura di $\{x_i\}_i \geq 1$, iniziamo ricordando che $(x \mod n) \mod q = x \mod q$ e quindi

$$\begin{aligned} z_{i+1} &= x_{i+1} \mod q && (41) \\ &= ((x_i^2 - 1) \mod n) \mod q && (42) \\ &= ((x_i \mod q)^2 - 1) \mod q = (z_i^2 - 1) \mod q && (43) \end{aligned}$$

Quindi ogni passo dell'algoritmo ha una controparte identica definita ombra. Ogni successione ombra è caratterizzata da un antiperiodo (λ) e un periodo (σ) le cui lunghezze sono $\mathcal{O}(\sqrt{q})$. Utilizzando la strategia di salvare valori ad intervalli sempre più grandi abbiamo la certezza che z_{2^i} salvato si trovi all'interno del periodo
NOTA: il fatto che $2^i > \sigma$ implica che esiste un $2^i < j \leq 2^{i+1}$ t.c. $z_j = z_{2^i}$
A questo punto ricordiamo che $z_i = x_i \mod q \forall i$ e dunque $x_j \mod q = x_{2^i} \mod q$ e di conseguenza $x_j - x_{2^i}$ è divisore di q e quindi di n. Il numero di passi per ottenere il più piccolo fattore primo p di n è $\mathcal{O}(\sqrt{p})$

Question 14

Casualità in crittografia?

In crittografia si utilizza sempre la casualità per non dare punti di riferimento ai malintenzionati, quindi i bit vengono generati casualmente, indipendenti e con probabilità uniforme

$$\text{prob}[x_{i+1} = 0 | x_i = 0] = \text{prob}[x_{i+1} = 0 | x_i = 1] = \text{prob}[x_i = 1] = \frac{1}{2}$$

Non è facile ottenere casualità se non al di fuori del PC, in particolare Linux utilizza `/dev/random` che utilizza l'entropia dei dispositivi per generare numeri casuali e blocca il processo se questa entropia non è sufficiente; esiste anche `/dev/urandom` che fa la stessa cosa ma non blocca il processo.
Esiste un tipo di algoritmo detto decisionale in cui l'output è binario (si/no, true/false,...) e

Question 14 continued on next page...

l'elemento di casualità può risiedere nel risultato (Monte Carlo) o nel tempo di calcolo (Las Vegas)

1. **Monte Carlo** ha le seguenti caratteristiche: se la risposta reale è TRUE allora restituisce TRUE con una probabilità indipendente dalla lunghezza dell'input ($\epsilon > 0$); se la risposta reale è FALSE restituisce sempre FALSE. Questo algoritmo è detto probability-bounded, ovvero che al crescere dell'input NON tende a 0. Un esempio di non-probability-bpunded è la procedura di determinazione del fatto che un numero n sia composto; questo perché sapendo che i divisori di n vanno in coppia (x e $\frac{n}{x}$) vuol dire che uno non è maggiore di $\sqrt{n}$ (i divisori di n sono al più $2\sqrt{n}$) quindi la probabilità che esso sia un divisore è
$\frac{2\sqrt{n}}{\frac{n}{4}} = 8\frac{\sqrt{n}}{n} = \frac{8}{\sqrt{n}}$ quindi tende a 0 al crescere di n
È inoltre importante limitare inferiormente l'errore probabilistico ϵ in modo tale che, su dei risultati indipendenti, si possa aumentare la probabilità di successo

2. **Las Vegas** restituisce sempre la risposta corretta (altrimenti non viene restituita) in un tempo dato da una certa probabilità. Un classico algoritmo di questo tipo è la procedura di generazione di una sorgente di bit uniformi (ovvero tali che prob$[x_i = 0] =$ prob$[x_i] = 1$) e segue questi passi

   (a) Si pone $i = 1$
   (b) Considera i bit z_i e z_{i+1}
   (c) Se $z_i = 0 e z_{i+1} = 1$ ritorna zero, se viceversa ritorna 1
   (d) Si incrementa i di due e torna al punto (b)

   Quindi la probabilità che il risultato sia restituito al k-esimo tentativo è data da $s^{k-1}(1-s)$ dove $s = 1 - 2p(1-p)$, questa è una nota distribuzione geometrica

$$\sum_{k=1}^{\infty} ks^{k-1}(1-s) = (1-s)\sum_{k=1}^{\infty} ks^{k-1} \tag{44}$$

$$= (1-s)\frac{d}{ds}\int_0^s (\sum_{k=1}^{\infty} kt^{k-1})dt = (1-s)\frac{d}{ds}\sum_{k=1}^{\infty}\int_0^s kt^{k-1}dt \tag{45}$$

$$= (1-s)\frac{d}{ds}\sum_{k=1}^{\infty} s^k \tag{46}$$

$$= (1-s)\frac{d}{ds}(\frac{1}{1-s} - 1) \tag{47}$$

$$= (1-s)\frac{1}{(1-s)^2} = \frac{1}{1-s} \tag{48}$$

   Dunque più è piccolo p e più il valore atteso diviene alto
   A questo punto ci si chiede quale sia la probabilità che $2k$ bit siano sufficienti per ottenere il risultato (probabilità che termini in k passi):
   $\sum_{i=1}^{k} s^{i-1}(1-s) = (1-\cancel{s}) + (\cancel{s} - \cancel{s^2}) + (\cancel{s^2} - \cancel{s^3}) + ... + \cancel{s^{k-1}} - s^k = 1 - s^k$

Question 15

El Gamal?

È un sistema crittografico strettamente legato a RSA e DH e funziona in questo modo

1. A determina p abbastanza lungo, $g \in Z_P^*$ e calcola $A = g^a \mod p$ dove $a \in Z_p^*$ è un numero casuale
2. A conserva a come chiave segreta ed invia (A, p, g) come chiave pubblica
3. B recupera la chiave pubblica di A
4. B sceglie un numero $k \in Z_p^*$ a caso e calcola $r = g^k \mod p$ e $t = (A^k \cdot M) \mod p$
5. Invia ad A la coppia $C = (r, t)$ come messaggio cifrato
6. A calcola $B = r^a \mod p$
7. Calcola (tramite EEUCLID) $B^{-1} \mod p$
8. Infine calcola $M = (B^{-1} \cdot t) \mod p$

La correttezza dipende dal fatto che $t \equiv (B \cdot M) \mod p$

$$t = (A^k \cdot M) \mod p \tag{49}$$
$$= ((g^a \mod p)^k \cdot M) \mod p = ((g^a)^k \cdot M) \mod p \tag{50}$$
$$= ((g^k)^a \cdot M) \mod p = ((g^k \mod p)^a \cdot M) \mod p \tag{51}$$
$$= (r^a \cdot M) \mod p \tag{52}$$
$$= ((r^a \mod p) \cdot M) \mod p = (B \cdot M) \mod p \tag{53}$$

L'efficienza è data dalla complessità nel: calcolare due potenze modulari (pre-computazione), calcolo di una potenza (decifrazione), un inverso moltiplicativo e una moltiplicazione. Quindi sono poche operazioni ma su migliaia di bit.
Violare ElGamal significa risolvere CDH (Computational Diffie–Hellman assumption), per cui bisogna stare attenti a non usare lo stesso valore k altrimenti, nel caso si riuscisse a decifrare M_1 si potrebbero decifrare anche tutti gli altri messaggi. Questo lo si può dimostrare con $C_1 = (r, t_1 = (A^k \cdot M_1) \mod p)$ e $C_2 = (r, t_2 = (A^k \cdot M_2) \mod p)$; moltiplicando $t_1^{-1} \cdot t_2 \mod p = (M_1^{-1} \cdot M_2) \mod p \Rightarrow M_2 = (M_1 \cdot (t_2 \cdot t_1^{-1})) \mod p$. Dunque CDH implica che ElGamal è sicuro poiché se si recupera M si può risalire a B